Name: Miheeka Kewalramani Roll No.45 Branch:AI&DS Class:T12 Subject:ASD&D

Experiment 03

Aim: To deploy multi-service applications using Docker Compose.

This experiment maps to LO4:

LO4: Containerization, orchestrate and deploy applications using Docker, Compose, Kubernetes and Cloud infrastructure tools.

Theory:

Docker Compose is a tool used to define and manage multi-container Docker applications. It allows multiple services like web servers, databases, and application servers to be configured in a single `docker-compose.yml` file and deployed together.

A **multi-service application** consists of multiple containers working together. For example:

- **Nginx** → Web server service
- **MySQL** → Database service

The `docker-compose.yml` file defines:

- **Services** – Containers required for the application
- **Images** – Docker images used to create containers
- **Ports** – Communication between host and containers
- **Environment variables** – Application configuration
- **Volumes** – Persistent data storage

Docker Compose automatically creates a network that allows services to communicate with each other. It simplifies deployment because all services can be started, stopped, and managed using simple commands like:

docker compose up -d
docker compose down

Advantages:

- Easy management of multiple containers
- Faster application deployment
- Consistent development environment
- Persistent storage using volumes

Thus, Docker Compose provides an efficient way to deploy and manage multi-service applications.

Common Docker Compose Commands

Command	Purpose
`docker compose up`	Creates and starts services
`docker compose up -d`	Runs services in background mode
`docker compose ps`	Displays running containers
`docker compose logs`	Shows service logs

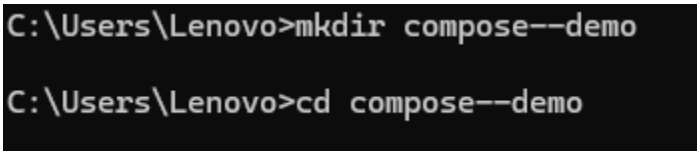
`docker compose stop`	Stops services
`docker compose start`	Starts stopped services
`docker compose restart`	Restarts containers
`docker compose down`	Removes containers and networks
`docker compose down -v`	Removes containers and volumes
`docker compose images`	Displays service images
`docker compose config`	Validates YAML configuration

Step 1: Create Project Directory

Open Command Prompt and execute:

mkdir compose-demo

cd compose-demo



Step 2: Create Docker Compose File

Create a file named:

docker-compose.yml

Add the following configuration:

```
version: '3.8'

services:

  web:
    image: nginx
    container_name: webserver
    ports:
      - "8081:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver
```

```
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123

    ports:
      - "3307:3306"

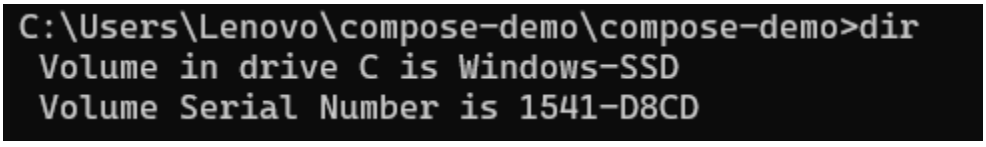
    volumes:
      - mysql-data:/var/lib/mysql

volumes:
  mysql-data:
```

Step 3: Verify Compose File

Command:

dir



Step 4: Start Multi-Service Application

Command:

docker compose up -d

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose up -d
[+] Running 2/3
 - Network compose-demo_default  Created
    0.9s
 ✔Container mysqlserver         Started
    0.9s
 ✔Container webserver           Started
    0.7s
```

Step 5: Check Running Services

Command:

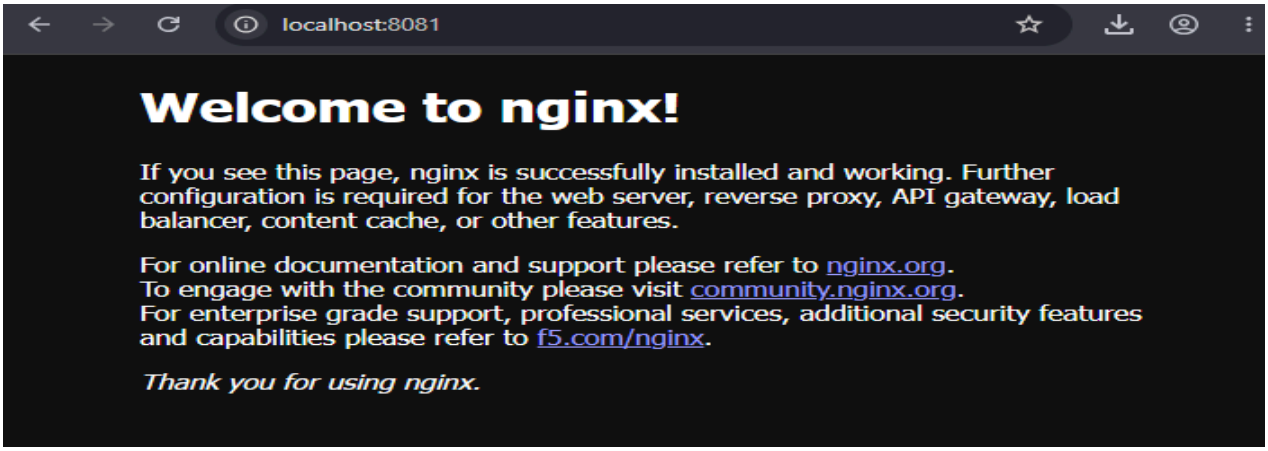
docker compose ps

```
C:\Users\Lenovo\compose-demo\compose-demo>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Step 6: Access Web Service

Open a browser and enter:

http://localhost:8081



The Nginx web server is successfully deployed.

Step 7: View Container Logs

Command:

docker compose logs database

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose logs database
mysqlserver  | 2026-07-28 05:06:21+00:00 [Note] [Entrypoint]: Entrypoint
 script for MySQL Server 8.0.46-1.el9 started.
mysqlserver  | 2026-07-28 05:06:22+00:00 [Note] [Entrypoint]: Switching
to dedicated user 'mysql'
mysqlserver  | 2026-07-28 05:06:22+00:00 [Note] [Entrypoint]: Entrypoint
 script for MySQL Server 8.0.46-1.el9 started.
mysqlserver  | 2026-07-28 05:06:22+00:00 [Note] [Entrypoint]: Initializi
ng database files
mysqlserver  | 2026-07-28T05:06:22.283058Z 0 [Warning] [MY-011068] [Serv
er] The syntax '--skip-host-cache' is deprecated and will be removed in
a future release. Please use SET GLOBAL host_cache_size=0 instead.
mysqlserver  | 2026-07-28T05:06:22.283617Z 0 [System] [MY-013169] [Serve
r] /usr/sbin/mysqld (mysqld 8.0.46) initializing of server in progress a
s process 81
mysqlserver  | 2026-07-28T05:06:22.302075Z 1 [System] [MY-013576] [InnoD
B] InnoDB initialization has started.
mysqlserver  | 2026-07-28T05:06:23.692945Z 1 [System] [MY-013577] [InnoD
B] InnoDB initialization has ended.
mysqlserver  | 2026-07-28T05:06:26.580115Z 6 [Warning] [MY-010453] [Serv
er] root@localhost is created with an empty password ! Please consider s
witching off the --initialize-insecure option.
mysqlserver  | 2026-07-28 05:06:34+00:00 [Note] [Entrypoint]: Database f
iles initialized
mysqlserver  | 2026-07-28 05:06:34+00:00 [Note] [Entrypoint]: Starting t
emporary server
```

Step 8: Verify Docker Images

Command:

docker compose images

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose images
CONTAINER           REPOSITORY          TAG                 IMAGE ID
        SIZE
mysqlserver         mysql               8.0                 6cd09145362d
        799MB
webserver           nginx               latest              4e5db4761e0f
        161MB
```

Step 9: Verify Docker Volume

Command:

docker volume ls

```
C:\Users\Lenovo\compose-demo\compose-demo>docker volume ls
DRIVER    VOLUME NAME
local     compose-demo_mysql-data
```

The volume provides persistent storage for MySQL data.

Step 10: Stop Application

Command:

docker compose down

```
C:\Users\Lenovo\compose-demo\compose-demo>docker compose down
[+] Running 3/3
 ✔Container webserver            Removed
 ✔Container mysqlserver          Remove...
 ✔Network compose-demo_default   Removed
```

This stops and removes the containers.

Conclusion:

I learned how to deploy a multi-service application using Docker Compose. I understood how to create a `docker-compose.yml` file, configure multiple services, manage containers, use Docker images, and store data using volumes. This experiment helped me understand how Docker Compose simplifies application deployment by managing multiple containers efficiently in a single configuration file.